

Kompresa: Statistické metody

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

05.06.2026

CC BY-NC-SA 4.0

Motivace

Proč komprimovat data?

Kolik dat generujeme každý den?

- 1 minuta videa ve 4K $\approx$ 375 MB (nekomprimovaně)
- 1 fotka z mobilu $\approx$ 25 MB (raw senzor)
- 1 stránka textu $\approx$ 2 KB

Bez komprese bychom potřebovali **mnohonásobně více** úložného prostoru a přenosové kapacity

Proč komprimovat data?

Jaké znáte formáty, které používají kompresi?

- komprese souborů – **ZIP, RAR, 7z**
- obrázky – **PNG, JPEG, WebP**
- zvuk – **MP3, FLAC, AAC**
- video – **H.264, H.265, AV1**
- webové stránky (HTTP komprese) – **gzip, brotli**

Ztrátová vs. bezztrátová komprese

Bezztrátová (*lossless*) – z komprimovaných dat lze **přesně** rekonstruovat originál

Ztrátová (*lossy*) – část informace se **zahodí** (typicky ta, kterou člověk nevnímá)

Ztrátová vs. bezztrátová komprese

Kde použít jakou metodu?

Typ dat	Bezztrátová	Ztrátová
Text / kód	ZIP, gzip	nelze
Obrázky	BMP, PNG	JPEG, WebP
Zvuk	WAV, FLAC	MP3, AAC
Video	–	H.264, AV1

Ztrátová vs. bezztrátová komprese

Proč **nelze** použít ztrátovou kompresi na text nebo zdrojový kód?

Protože **každý bit** nese důležitou informaci

- změna jediného znaku může změnit celý význam

Dnes se budeme zabývat výhradně **bezztrátovou** kompresí

RLE (Run-Length Encoding)

Nejjednodušší kompresní metoda

Představte si obrázek s velkými jednobarevnými plochami – proč ukládat každý pixel zvlášť?

Místo opakování stejné hodnoty uložíme **hodnotu a počet opakování**

AAAAAABBBCCDDDDDDDDDD

Zakódujeme jako:

6A3B2C8D

Z 19 znaků na 8 znaků – kompresní poměr $\approx 42\%$

Kdy RLE funguje dobře?

Funguje skvěle na datech s dlouhými sekvencemi opakování

- Jednoduché obrázky (ikony, diagramy, loga)
- Faxové přenosy (standard CCITT) – většina řádku je bílá
- Run-length kódování je součástí formátů BMP, TIFF, PCX

Kdy naopak selže?

ABCDEFGHIJKLMNOP

1A1B1C1D1E1F1G1H1I1J1K1L1M1N1O1P

Z 16 znaků na 32 znaků – data se **zdvojnásobila**

Implementace v C

```
void rle_encode(const char *input, char *output) {  
    int i = 0, j = 0;  
    while (input[i] != '\0') {  
        char current = input[i];  
        int count = 1;  
        while (input[i + count] == current)  
            count++;  
        j += sprintf(output + j, "%d%c", count, current);  
        i += count;  
    }  
    output[j] = '\0';  
}
```

Implementace v C

Jaká je složitost RLE kódování?

Časová: $\mathcal{O}(n)$ – projdeme vstup jednou

Prostorová: $\mathcal{O}(n)$ – v nejhorším případě $2\times$ větší výstup

RLE je **jednoduché a rychlé**, ale funguje pouze na specifických datech
– potřebujeme něco chytřejšího

Entropie a informace

Kolik informace data skutečně nesou?

Představte si, že házete **mincí** – kolik informace získáte jedním hodem?

Dva stejně pravděpodobné výsledky $\rightarrow$ potřebujeme **1 bit** (0 nebo 1)

Jinými slovy: 1 bit je množství informace, které potřebujeme k rozlišení **dvou stejně pravděpodobných** možností

A co **kostkou**? 6 výsledků $\rightarrow \log_2 6 \approx 2.58$ bitů

Potřebujeme víc než 2 bity, ale méně než 3

- se 2 bity rozlišíme jen 4 možnosti, se 3 bity už 8

Kolik informace data skutečně nesou?

Obecně: výsledek s pravděpodobností p nese

$$-\log_2 p$$

bitů informace

Čím méně pravděpodobný výsledek, tím více informace nese

Kolik informace data skutečně nesou?

Proč $-\log_2 p$?

- Pravděpodobnost $p = 1$ (jistý výsledek) $\rightarrow -\log_2 1 = 0$ bitů
 - žádné překvapení, žádná informace
- Pravděpodobnost $p = \frac{1}{2}$ $\rightarrow -\log_2(2^{-1}) = -(-1) = 1$ bit
- Pravděpodobnost $p = \frac{1}{256}$ $\rightarrow -\log_2(2^{-8}) = -(-8) = 8$ bitů
 - velmi překvapivý výsledek, hodně informace

Shannonova entropie

Claude Shannon (1948) definoval **entropii** jako průměrnou informaci na symbol:

$$H = - \sum_{i=1}^n p_i \cdot \log_2 p_i$$

kde p_i je pravděpodobnost i -tého symbolu

Entropie udává **teoretické minimum**

- průměrný počet bitů na symbol, pod který se nelze dostat

Shannonova entropie

Příklad: text obsahující pouze znaky A, B, C, D s frekvencemi:

Symbol	A	B	C	D
Frekvence	50%	25%	12.5%	12.5%

$$H = -\left(\frac{1}{2} \cdot \log_2 2^{-1} + \frac{1}{4} \cdot \log_2 2^{-2} + \frac{1}{8} \cdot \log_2 2^{-3} + \frac{1}{8} \cdot \log_2 2^{-3}\right)$$

$$H = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bitů/symbol}$$

Shannonova entropie

Kdybychom kódovali tyto 4 symboly v ASCII, potřebujeme **8 bitů** na znak

Ale právě jsme spočítali, že entropie je jen $H = 1.75$ bitů/symbol

Většina z těch 8 bitů je **redundantní** – nenese žádnou novou informaci

Kompresní poměr: $\frac{1.75}{8} \approx 22\%$

- teoreticky můžeme zmenšit data na pětinu

Žádný bezztrátový algoritmus nemůže překonat Shannonův limit

Ale dá se ho dosáhnout?

Huffmanovo kódování

Prefixové kódy

Chceme přiřadit symbolům kódy **různé délky** – častější symboly dostanou **kratší** kódy

Proč nemůžeme jednoduše přiřadit $A=0$, $B=1$, $C=10$, $D=11$?

Jak dekódujeme 010? Je to **AC** (0, 10) nebo **ABA** (0, 1, 0)?

Podmínka prefixových kódů: žádný kód nesmí být **prefixem** (začátkem) jiného kódu

Takovým kódům říkáme **prefixové kódy** (*prefix-free codes*)

Huffmanův algoritmus

David Huffman (1952) vymyslel **optimální** způsob, jak přiřadit prefixové kódy

Algoritmus:

1. Vytvoř list (stromu) pro každý symbol s jeho frekvencí
2. Vyber **dva uzly s nejmenší frekvencí**
3. Spoj je pod nový rodičovský uzel (součet frekvencí)
4. Opakuj od kroku 2, dokud nezbyde **jeden kořen**

Výsledkem je **binární strom** – levá hrana = 0, pravá hrana = 1

Stavba Huffmanova stromu

Krok 1 – vytvoříme list pro každý symbol s jeho frekvencí:

A
50%

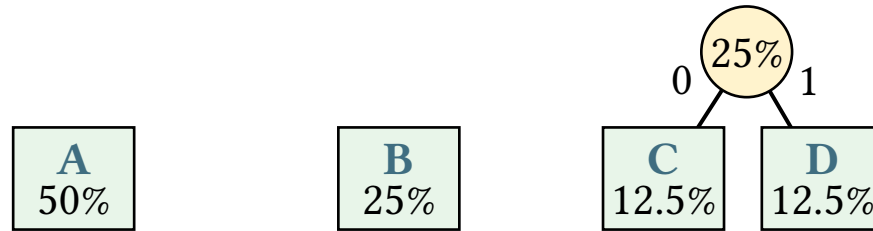
B
25%

C
12.5%

D
12.5%

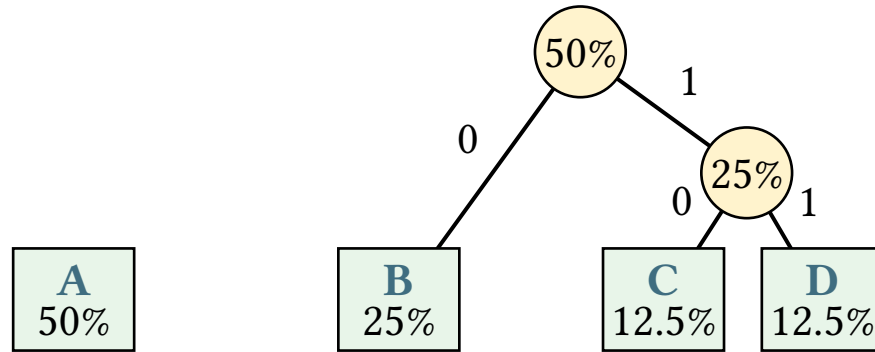
Stavba Huffmanova stromu

Kroky 2–3, iterace 1: vybereme C a D (nejmenší frekvence), spojíme je:



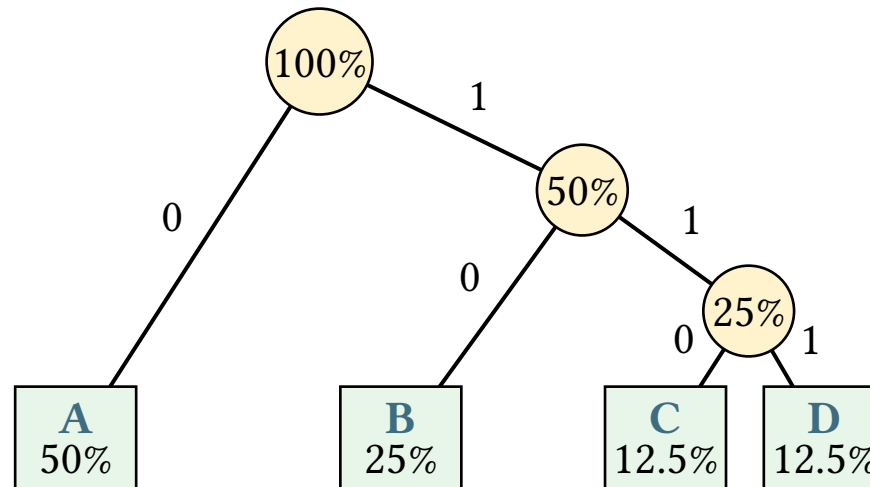
Stavba Huffmanova stromu

Kroky 2–3, iterace 2: vybereme B (25%) a uzel CD (25%), spojíme je:



Stavba Huffmanova stromu

Kroky 2–3, iterace 3: spojíme A (50%) a uzel BCD (50%) – **krok 4**:
zbývá jeden kořen:



Výsledné kódy

Kód symbolu = cesta od kořene k listu:

Symbol	Frekvence	Huffmanův kód	Počet bitů
A	50%	0	1
B	25%	10	2
C	12.5%	110	3
D	12.5%	111	3

Výsledné kódy

Průměrná délka kódu (L):

$$L = \sum p_i \cdot l_i$$

kde l_i je délka kódu i -tého symbolu

$$L = 0.5 \cdot 1 + 0.25 \cdot 2 + 0.125 \cdot 3 + 0.125 \cdot 3 = 1.75 \text{ bitů/symbol}$$

Průměrná délka kódu se shoduje s entropií ($H = 1.75$ bitů/symbol)!

Huffmanovo kódování vždy generuje **optimální** prefixový kód

Zakódování zprávy

Zakódujme zprávu AABACDA:

A	A	B	A	C	D	A
0	0	10	0	110	111	0

Výsledek: 001001101110 – 12 bitů místo $7 \cdot 8 = 56$ bitů (ASCII)

- **kompresní poměr** $\frac{12}{56} \approx 21.4\%$

Dekódování: čteme bit po bitu a procházíme stromem od kořene

- **jakmile dojdeme na list**, máme symbol

Redundance

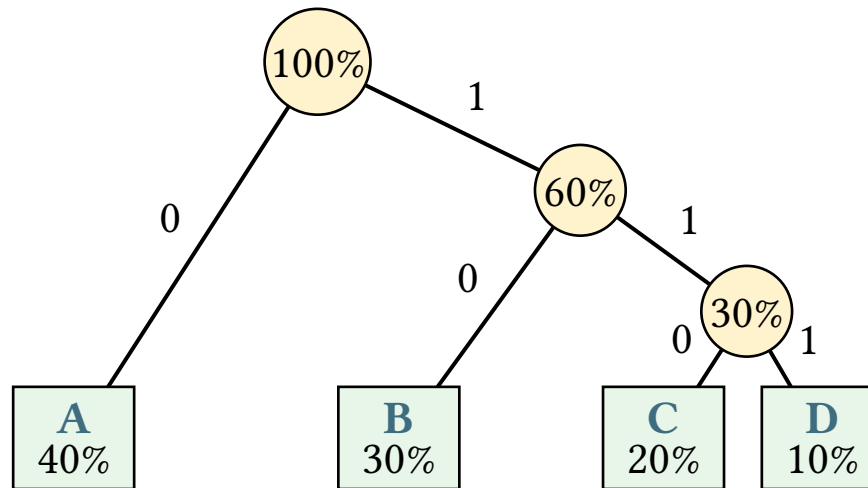
V předchozím příkladu byly frekvence mocninami $\frac{1}{2} \rightarrow$ ideální počet bitů ($-\log_2 p_i$) pro každý symbol vychází **celočíslně** $\rightarrow$ Huffman nepotřebuje zaokrouhlovat $\rightarrow L = H = 1.75$

Co když frekvence nebudou mocniny dvou?

Symbol	A	B	C	D
Frekvence	40%	30%	20%	10%

Redundance

Huffmanův strom pro frekvence 40%, 30%, 20%, 10%:



Redundance

Symbol	Frekvence	Huffmanův kód	Počet bitů	$-p_i \cdot \log_2 p_i$
A	40%	0	1	≈ 0.529
B	30%	10	2	≈ 0.521
C	20%	110	3	≈ 0.464
D	10%	111	3	≈ 0.332

$$L = 0.4 \cdot 1 + 0.3 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 3 = 1.9 \text{ bitů/symbol}$$

$$H \approx 0.529 + 0.521 + 0.464 + 0.332 \approx 1.846 \text{ bitů/symbol}$$

Redundance

$L = 1.9 > H \approx 1.846$ – Huffman **nedosáhl** Shannonova limitu!

Pro Huffmanovo kódování platí:

$$H(X) \leq L < H(X) + 1$$

$$1.846 \leq 1.9 < 2.846 \quad \checkmark$$

Redundance kódu: $R = L - H \approx 1.9 - 1.846 = 0.054$ bitů/symbol

Čím blíže jsou frekvence mocninám $\frac{1}{2}$, tím menší redundance

Huffmanovo kódování v praxi

Proč se téměř nikdy nepoužívá samostatně?

Huffmanovo kódování komprimuje data jen na úrovni **jednotlivých symbolů** – nevyužívá opakující se vzory v datech (např. the v angličtině)

Proto se téměř vždy kombinuje s jinou metodou a je použito pouze jako **závěrečný krok** transformace

Kde se Huffmanovo kódování používá?

- **ZIP** – kombinace s LZ77 (deflate algoritmus)
- **JPEG** – Huffmanovo kódování kvantizovaných koeficientů
- **MP3** – kódování frekvenčních koeficientů
- **PNG** – kombinace s LZ77 (také deflate)

Aritmetické kódování

Aritmetické kódování

Huffmanovo kódování přiřazuje každému symbolu **celočíselný** počet bitů (1, 2, 3, ...)

Co když entropie říká, že optimální délka kódu pro symbol je **1.3 bitu**?

Huffman musí zaokrouhlit na 1 nebo 2 bity

- nikdy nemůže být zcela optimální (redundance)

Aritmetické kódování toto omezení nemá

- kóduje celou zprávu jako **jedno číslo**

Princip

Začneme s intervalem $[0, 1)$

Pro každý symbol **zúžíme** interval podle jeho pravděpodobnosti

Symbole A (60%), B (20%), C (20%) – rozdělení intervalu:

Symbol	Pravděpodobnost	Podinterval
A	60%	$[0, 0.6)$
B	20%	$[0.6, 0.8)$
C	20%	$[0.8, 1.0)$

Algoritmus kódování

Máme aktuální interval $[l, h)$ a symbol s podintervalem $[c_l, c_h)$:

$$l_{\text{new}} = l + (h - l) \cdot c_l$$

$$h_{\text{new}} = l + (h - l) \cdot c_h$$

$(h - l)$ je **šířka** aktuálního intervalu – nový interval je jeho proporcionální část

Čím méně pravděpodobný symbol, tím víc se interval **zúží** – potřebujeme více bitů k jeho reprezentaci

Algoritmus kódování

Zakódujme zprávu BAC – začínáme intervalem $[l, h) = [0, 1)$

B $[c_l, c_h) = [0.6, 0.8)$:

$$l = 0 + 1 \cdot 0.6 = 0.6, \quad h = 0 + 1 \cdot 0.8 = 0.8$$

A $[c_l, c_h) = [0, 0.6)$:

$$l = 0.6 + 0.2 \cdot 0 = 0.6, \quad h = 0.6 + 0.2 \cdot 0.6 = 0.72$$

C $[c_l, c_h) = [0.8, 1.0)$:

$$l = 0.6 + 0.12 \cdot 0.8 = 0.696, \quad h = 0.6 + 0.12 \cdot 1.0 = 0.72$$

Algoritmus kódování

Celou zprávu BAC reprezentujeme **libovolným číslem** z $[0.696, 0.72)$ – například 0.7

Algoritmus dekódování

Máme zakódované číslo x a původní tabulku podintervalů:

1. Najdeme symbol, jehož podinterval $[c_l, c_h)$ obsahuje x
2. **Přeškálujeme** x zpět na $[0, 1)$:

$$x_{\text{new}} = \frac{x - c_l}{c_h - c_l}$$

Přeškálování „odstraní“ dekódovaný symbol – zbytek čísla kóduje
zbytek zprávy

Opakujeme, dokud nedekódujeme všechny symboly

Algoritmus dekódování

Dekódujme číslo 0.7 – symboly A $[0, 0.6)$, B $[0.6, 0.8)$, C $[0.8, 1.0)$

$$0.7 \in [0.6, 0.8) \rightarrow \mathbf{B} \quad x = \frac{0.7-0.6}{0.8-0.6} = 0.5$$

$$0.5 \in [0, 0.6) \rightarrow \mathbf{A} \quad x = \frac{0.5-0}{0.6-0} = 0.833\dots$$

$$0.833\dots \in [0.8, 1.0) \rightarrow \mathbf{C}$$

Výsledek: BAC

Co by se stalo, kdybychom pokračovali v dekódování?

- algoritmus by vesele generoval další symboly „do nekonečna“

Jak poznat, kdy přestat dekódovat?

1. **Uložit délku zprávy** zvlášť – dekodér předem ví, kolik symbolů má dekódovat
2. **Speciální koncový symbol** (EOF) – přidáme do abecedy nový symbol – dekodér skončí, jakmile ho přečte

Jak poznat, kdy přestat dekódovat?

Jakou pravděpodobnost přiřadit EOF?

- přidání symbolu **vždy změni** rozložení pravděpodobností

EOF dostane **velmi malou pravděpodobnost** – čím delší zpráva, tím menší podíl EOF zabere a tím méně ovlivní kompresi ostatních symbolů

Shrnutí

Výhody aritmetického kódování:

- Dosahuje kompresního poměru **blíže entropii** než Huffman
- Zvláště efektivní, když pravděpodobnosti nejsou mocniny $\frac{1}{2}$

Nevýhody:

- Složitější implementace
- Problémy s přesností desetinných čísel
 - v praxi se používá **celočíselná aritmetika**
- Historicky patentované (patenty již vypršely)

Celočíselná aritmetika

V našem příkladu jsme pracovali s desetinnými čísly: 0.696, 0.72, 0.833...

Problém: s každým symbolem se interval zužuje

→ potřebujeme stále více desetinných míst

→ float ani double nestačí

Řešení: interval $[0, 1)$ nahradíme celočíselným rozsahem

- např. $[0, 2^{32})$

Od pravděpodobností k četnostem

V desetinné verzi jsme pracovali s pravděpodobnostmi

V celočíselné verzi místo nich použijeme **tabulku četností**:

Symbol	Četnost	C_l	C_h	Význam
A	3	0	3	symboly před A: žádné
B	1	3	4	před B: $3 \times A$
C	1	4	5	před C: $3 \times A + 1 \times B$

$$T = 3 + 1 + 1 = 5$$

- C_l je **kumulativní četnost** symbolů před aktuálním, $C_h = C_l + \text{četnost}$

Vzorec pro zúžení intervalu

$$l_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_l}{T} \right\rfloor$$
$$h_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_h}{T} \right\rfloor - 1$$

Vše je **celočíselné** – násobení a dělení celých čísel, žádné floaty

Poměr $\frac{C_l}{T}$ a $\frac{C_h}{T}$ nahrazuje původní pravděpodobnosti

- např. pro A: $\frac{C_l}{T} = \frac{0}{5} = 0$, $\frac{C_h}{T} = \frac{3}{5} = 0.6$ – totéž jako dříve

Průběh algoritmu

Průběžně **vypouštíme hotové bity**:

1. Po každém zúžení intervalu zkontroluj MSB l a h
2. Pokud se shodují $\rightarrow$ ten bit se už **nemůže změnit**
 - zapiš ho na výstup
 - „roztáhni“ interval: odstraň MSB a přidej 0 k l , 1 k h
 - opakuj od kroku 1
3. Pokud se liší $\rightarrow$ **zatím nelze rozhodnout**, pokračuj dalším symbolem

Kódování pomocí celočíselného kódování

Stejná zpráva BAC jako dříve, A (60%), B (20%), C (20%) a 8-bitový registr

$$A : \left[\underbrace{00000000}_0, \underbrace{10011000}_{152} \right]$$

$$B : \left[\underbrace{10011001}_{153}, \underbrace{11001011}_{203} \right]$$

$$C : \left[\underbrace{11001100}_{204}, \underbrace{11111111}_{255} \right]$$

Kódování pomocí celočíselného kódování

Počáteční stav: $l = 0$, $h = 255$

B ($C_l = 3$, $C_h = 4$, $T = 5$):

$$l_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_l}{T} \right\rfloor = 0 + \left\lfloor \frac{256 \cdot 3}{5} \right\rfloor = 153 = 10011001$$

$$h_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_h}{T} \right\rfloor - 1 = 0 + \left\lfloor \frac{256 \cdot 4}{5} \right\rfloor - 1 = 203 = 11001011$$

MSB shodné (oba 1)

→ **výstup 1**, roztažení: $l = 50 = 00110010$, $h = 151 = 10010111$

Kódování pomocí celočíselného kódování

Stav po B: $l = 50$, $h = 151$

A ($C_l = 0$, $C_h = 3$, $T = 5$):

$$l_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_l}{T} \right\rfloor = 50 + \left\lfloor \frac{102 \cdot 0}{5} \right\rfloor = 50 = 00110010$$

$$h_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_h}{T} \right\rfloor - 1 = 50 + \left\lfloor \frac{102 \cdot 3}{5} \right\rfloor - 1 = 110 = 01101110$$

MSB shodné (oba 0)

→ **výstup 0**, roztažení: $l = 100 = 01100100$, $h = 221 = 11011101$

Kódování pomocí celočíselného kódování

Stav po A: $l = 100$, $h = 221$

C ($C_l = 4$, $C_h = 5$, $T = 5$):

$$l_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_l}{T} \right\rfloor = 100 + \left\lfloor \frac{122 \cdot 4}{5} \right\rfloor = 197 = \text{11000101}$$

$$h_{\text{new}} = l + \left\lfloor \frac{(h - l + 1) \cdot C_h}{T} \right\rfloor - 1 = 100 + \left\lfloor \frac{122 \cdot 5}{5} \right\rfloor - 1 = 221 = \text{11011101}$$

Kódování pomocí celočíselného kódování

MSB shodné (oba 1)

→ **výstup 1**, roztažení: $l = 10001010$, $h = 10111011$

MSB shodné (oba 1)

→ **výstup 1**, roztažení: $l = 00010100$, $h = 01110111$

MSB shodné (oba 0)

→ **výstup 0**, roztažení: $l = 00101000$, $h = 11101111$

MSB různé → stop

Kódování pomocí celočíselného kódování

Výstup: **10110...** – to je binární zápis čísla z intervalu $[0.696, 0.72)$

- shoduje se s desetinným příkladem!

Kódování pomocí celočíselného kódování

Co když se l a h „zaseknou“ těsně pod a nad polovinou?

Např. $l = 01111\dots$ a $h = 10000\dots$ – MSB se liší, ale interval je **velmi úzký**

Hrozí, že nám dojde přesnost dřív, než se MSB shodnou

Řešení – E3 škálování (*underflow handling*):

- odstraníme **druhý** bit, zapamatujeme si, že jsme to udělali
- jakmile se MSB konečně shodne, zapíšeme ho + **obrácené kopie** za každé E3 škálování

Dekódování celočíselného kódování

Dekodér udržuje stejné l , h jako kodér + **buffer** v naplněný bity ze vstupu

1. Spočítej pozici v intervalu: $s = \left\lfloor \frac{(v-l+1) \cdot T - 1}{h-l+1} \right\rfloor$
2. Najdi symbol, jehož kumulativní rozsah $[C_l, C_h)$ obsahuje s
3. Zúži l , h stejným vzorcem jako kodér
4. Při shodě MSB proved' roztažení l , h a do v **načti další bit** ze vstupu
5. Opakuj od kroku 1

Kodér bity **zapisuje** $\rightarrow$ dekodér je **čte** – oba udržují synchronní $[l, h)$

Dekódování aritmetického kódování

Dekódujme bitový proud 10110001... – v naplníme prvními 8 bity: $v = 177$

Počáteční stav: $l = 0$, $h = 255$

$$s = \left\lfloor \frac{(v-l+1) \cdot T - 1}{h-l+1} \right\rfloor = \left\lfloor \frac{(177-0+1) \cdot 5 - 1}{256} \right\rfloor = \left\lfloor \frac{889}{256} \right\rfloor = 3$$

$$s = 3 \in [3, 4) \rightarrow \mathbf{B} (C_l = 3, C_h = 4)$$

$$l_{\text{new}} = 0 + \left\lfloor \frac{256 \cdot 3}{5} \right\rfloor = 153 = 10011001$$

$$h_{\text{new}} = 0 + \left\lfloor \frac{256 \cdot 4}{5} \right\rfloor - 1 = 203 = 11001011$$

Dekódování aritmetického kódování

MSB shodné (oba 1) $\rightarrow$ odstraníme MSB, posuneme vlevo, l doplníme 0, h doplníme 1, do v načteme bit:

$$l : \textcolor{red}{1}0011001 \rightarrow 0011001\textcolor{teal}{0} = 50$$

$$h : \textcolor{red}{1}1001011 \rightarrow 1001011\textcolor{teal}{1} = 151$$

$$v : \textcolor{red}{1}0110001 \rightarrow 0110001\textcolor{teal}{0} = 98$$

Dekódování aritmetického kódování

Stav: $l = 50$, $h = 151$, $v = 98$

$$s = \left\lfloor \frac{(98-50+1) \cdot 5 - 1}{102} \right\rfloor = \left\lfloor \frac{244}{102} \right\rfloor = 2 \rightarrow \mathbf{A} (C_l = 0, C_h = 3)$$

$$l_{\text{new}} = 50 + \left\lfloor \frac{102 \cdot 0}{5} \right\rfloor = 50 = \mathbf{00110010}$$

$$h_{\text{new}} = 50 + \left\lfloor \frac{102 \cdot 3}{5} \right\rfloor - 1 = 110 = \mathbf{01101110}$$

Dekódování aritmetického kódování

MSB shodné (oba 0) $\rightarrow$ stejná operace:

$$l : \text{ } 00110010 \rightarrow 01100100 = 100$$

$$h : \text{ } 01101110 \rightarrow 11011101 = 221$$

$$v : \text{ } 01100010 \rightarrow 11000101 = 197$$

Dekódování aritmetického kódování

Stav: $l = 100, h = 221, v = 197$

$$s = \left\lfloor \frac{(197-100+1) \cdot 5 - 1}{122} \right\rfloor = \left\lfloor \frac{489}{122} \right\rfloor = 4 \rightarrow \mathbf{C}$$

Výsledek: **BAC** – shoduje se s původní zprávou

Použití aritmetického kódování

Používá se v **JPEG 2000**, **HEIF**, **H.265/HEVC**, **VP9**

Shrnutí

Shrnutí

- **Komprese** snižuje velikost dat odstraněním **redundance**
- **RLE** – nejjednodušší metoda, počítá opakování
- **Shannonova entropie** – teoretický limit komprese, pod který se nelze dostat
- **Huffmanovo kódování** – optimální prefixové kódy na základě frekvence
- **Aritmetické kódování** – kóduje celou zprávu jako jedno číslo, blíže entropii
 - celočíselná varianta řeší problémy s přesností